

CymBuild AI Assistant

Product vision

Create a premium AI assistant for CymBuild that helps users learn the system, navigate workflows, troubleshoot issues, and complete tasks with confidence.

The product should feel like two things working together:

1. **An in-product copilot** for getting help while using CymBuild.
2. **A standalone help center** for onboarding, workflow guidance, knowledge discovery, and reusable playbooks.

The assistant should reduce training time, lower support load, improve consistency, and help users move from confusion to action quickly.

Core goals

- Help CymBuild users get unstuck fast.
- Explain CymBuild workflows in plain language.
- Turn natural language questions into practical next steps.
- Surface trusted internal knowledge with clear source attribution.
- Preserve conversation history, playbooks, bookmarks, and workflow progress.
- Give admins visibility into content gaps and answer quality.
- Fit naturally with CymBuild's metadata-driven architecture and enterprise security model.

Target users

General users

Need simple explanations, guided walkthroughs, and help with day-to-day tasks.

Project managers / delivery users

Need workflow help, reporting help, troubleshooting, permissions understanding, and operational guidance.

Finance / admin / advanced users

Need fast expert answers, reusable operational playbooks, and deeper workflow support.

Platform admins

Need knowledge management, analytics, quality review, and workflow curation.

Product pillars

1. Learn

Teach users what CymBuild does, how key modules fit together, and what steps to take.

2. Navigate

Help users find the right page, tab, workflow, action, and next move.

3. Do

Guide users through real tasks such as onboarding, project setup, reporting, issue resolution, and feature discovery.

4. Improve

Capture analytics, failed answers, feedback, and content gaps so admins can continuously improve the assistant.

Experience overview

Public landing page

No authentication required.

Purpose

Explain what the assistant is, who it helps, and why it saves time.

Sections

- Hero headline: "Your CymBuild copilot"
- Short summary of what it does
- Primary CTAs: **Sign up** and **Log in**
- Benefits cards:
 - Learn CymBuild faster
 - Get guided workflow help
 - Troubleshoot with confidence
 - Use trusted internal knowledge
- Feature showcase
- Screenshot and document help section
- Admin knowledge management section
- FAQ section
- Footer with product / support links

Tone

Modern, calm, premium, and accessible to non-technical construction teams.

Auth flow

Landing page is public.

Authentication is required only when the user clicks:

- Sign up
- Log in
- Open assistant workspace
- Save bookmarks
- Use uploads / history / admin tools

Auth requirements

- Support standard CymBuild identity approach
- Role-aware post-login experience
- Admin capabilities hidden unless the user has relevant permissions

Post-login dashboard

After login, the user lands in the assistant workspace.

Main layout

Three-column desktop layout:

Left rail

- New chat
- Saved conversations
- Bookmarks
- Workflow playbooks
- Knowledge base
- Recent topics
- Admin (if allowed)

Center panel

- Main chat area
- Suggested starter prompts
- Mode toggle: Beginner / Expert
- Workflow selector
- Streaming assistant responses

- Source citations
- Follow-up suggestions
- Feedback controls

Right rail

Context and utility area:

- Quick actions
- Current workflow checklist
- Related articles
- Relevant screenshots / uploads
- Recent help topics
- Bookmarked answers

Mobile layout

- Bottom navigation or top segmented nav
- Chat-first design
- Drawers for saved chats, bookmarks, and admin tools

Main assistant features

1. Natural language CymBuild help

Users can ask questions like:

- How do I create a new enquiry?
- Why can't I send this quote?
- What does this workflow status mean?
- How do I find invoice automation issues?
- Where do I manage permissions?

Assistant response style

Every answer should be structured as appropriate using one or more of:

- Plain language explanation
- Step-by-step guidance
- What this means
- What to do next
- Common mistakes
- Related follow-up questions
- Trusted sources used

Modes

Beginner mode

- Less jargon
- More explanation
- More step-by-step guidance
- More warnings and common pitfalls

Expert mode

- Concise
- Faster answers
- Assumes familiarity with CymBuild terms
- Prioritises actions, shortcuts, and direct outcomes

2. Guided workflows

Users can choose a predefined workflow and receive structured help.

Example workflow categories

- Onboarding
- Project setup
- Reporting
- Troubleshooting
- Permissions and access

- Finance operations
- Quote and enquiry lifecycle
- Job delivery workflows
- Feature discovery

Flow

1. User picks workflow.
2. Assistant asks clarifying questions if needed.
3. Assistant generates a guided plan.
4. User sees steps/checklist.
5. User can save as a reusable playbook.

Output formats

- Guided walkthrough
- Checklist
- Short how-to
- Decision-tree style help
- Troubleshooting path

3. Knowledge base

A dedicated searchable area for trusted content.

Content types

- FAQs
- Workflow guides
- SOPs
- Admin-written playbooks
- PDFs
- Product docs
- Screenshots
- Release notes
- Troubleshooting guides

User experience

- Search bar with filters
- Category browse
- Most viewed / recently updated
- Featured workflows
- Related content suggestions

Answer grounding

The assistant should use uploaded knowledge content when available and clearly show sources for each answer.

4. Screenshot help

Users can upload screenshots from CymBuild.

Assistant should be able to

- Describe what screen is shown
- Explain important sections
- Identify likely buttons or menu paths
- Point out missing setup or common mistakes
- Suggest what to do next

UX pattern

- Upload screenshot inline in chat
- Assistant returns:
 - What you are looking at
 - What matters on this screen

- Likely next step
- Common mistakes to check

Optional enhancement

- Clickable overlays / highlighted regions in future phase

5. Bookmarks and playbooks

Users should be able to save useful content.

Bookmarking

- Save an answer
- Add label / tag
- Find later from bookmarks area

Playbooks

- Save a conversation or answer as a reusable workflow
- Admin-created playbooks can be featured globally
- User-created playbooks remain user-scoped unless promoted

6. Suggested follow-ups

For every answer, suggest relevant next questions such as:

- Do you want the exact screen path?
- Would you like a checklist version?
- Do you want troubleshooting steps?
- Do you want the admin version of this process?

7. Feedback and answer quality

Every answer should support:

- Helpful / not helpful
- Optional comment
- Mark as incomplete
- Request content improvement

This feeds admin analytics and content gap tracking.

Admin experience

Admin dashboard

Admins need a management area for assistant operations.

Sections

Users and roles

- View assistant users
- Manage assistant roles
- See activity by role / team

Knowledge content

- Upload docs, screenshots, SOPs, PDFs
- Replace existing versions
- Organise categories and tags
- Mark authoritative sources
- Feature content

Workflow library

- Manage prebuilt prompts
- Publish admin workflow guides
- Promote successful user playbooks

Analytics

- Most common questions

- Top searched topics
- Failed answers
- Low-confidence answers
- Content gaps
- Most helpful sources
- Screenshot-help trends

Quality review

- Review conversations
- Inspect poor feedback cases
- Improve prompts / routing / knowledge content

Information architecture

Primary pages

1. Landing page
2. Login / sign-up
3. Assistant dashboard
4. Conversation detail
5. Bookmarks
6. Playbooks
7. Knowledge base
8. Knowledge article detail
9. Upload manager
10. Admin dashboard
11. Analytics
12. Workflow library
13. Settings / preferences

Recommended page breakdown

Landing page

- Hero
- Benefits
- How it works
- Example use cases
- Knowledge trust section
- Screenshot help section
- FAQ
- CTA

Assistant dashboard

- Search / ask anything input
- Mode toggle
- Suggested prompts
- Recent topics
- Saved conversations list
- Chat window

Knowledge base

- Search
- Filters
- Categories
- Featured guides
- Recent updates

Admin content manager

- Upload

- Replace
- Categorise
- Tag
- Activate / deactivate
- Source priority rules

Recommended technical architecture for CymBuild

Front end

Blazor WebAssembly PWA aligned to the CymBuild platform.

Main UI areas

- Public landing experience
- Authenticated assistant workspace
- Admin management area
- Shared UI components in Concursus.Components.Shared
- Styling via CymBuild standard stylesheet

Front-end patterns

- Responsive layout
- Streaming chat UX
- Drawer-based mobile navigation
- Progressive loading for knowledge and history
- Metadata-friendly admin grids and forms where possible

API layer

gRPC services in Concursus_API.

Suggested service areas

- AssistantChatService
- AssistantConversationService
- AssistantKnowledgeService
- AssistantWorkflowService
- AssistantBookmarkService
- AssistantAdminService
- AssistantAnalyticsService
- AssistantVisionService

Responsibilities

- Chat orchestration
- Conversation persistence
- Knowledge retrieval
- Source citation packaging
- Role-aware behavior
- Upload registration and content ingestion
- Analytics capture
- Screenshot analysis requests

Data layer

SQL Server with CymBuild conventions.

Critical platform rules to follow

- Every major entity must have a corresponding SCore.DataObjects row.
- EntityTypeeld must be set at insert time.
- No insert-then-update identity workaround.
- Workflow status must use SCore.DataObjectTransitionUpsert where workflow exists.
- UI behaviour should stay metadata-driven where that is appropriate.

Suggested database modules

Create a dedicated schema for the assistant, for example:

- SAi

Suggested entities

- AssistantUsers
- AssistantConversations
- AssistantMessages
- AssistantBookmarks
- AssistantPlaybooks
- AssistantPlaybookSteps
- AssistantKnowledgeItems
- AssistantKnowledgeItemVersions
- AssistantKnowledgeCategories
- AssistantKnowledgeTags
- AssistantKnowledgeSources
- AssistantUploads
- AssistantWorkflowTemplates
- AssistantWorkflowRuns
- AssistantWorkflowRunSteps
- AssistantFeedback
- AssistantAnalyticsEvents
- AssistantTopicClusters
- AssistantContentGaps
- AssistantRoleProfiles

Persistence rule

All user-owned tables should include a UserId column as requested, but still follow CymBuild identity rules where the row itself is a major entity.

Recommended data model summary

AssistantConversations

Stores a chat thread.

Fields:

- ID
- Guid
- UserId
- Title
- Mode
- LastActivityUtc
- IsArchived
- RowStatus
- RowVersion

AssistantMessages

Stores individual messages.

Fields:

- ID
- Guid
- ConversationId
- UserId
- MessageRole
- Content
- ContentFormat
- SourcePayloadJson
- ConfidenceScore
- CreatedUtc

- RowStatus
- RowVersion

AssistantBookmarks

Stores saved answers.

Fields:

- ID
- Guid
- UserId
- ConversationId
- MessageId
- Title
- Notes
- CreatedUtc
- RowStatus
- RowVersion

AssistantKnowledgeItems

Represents a logical knowledge asset.

Fields:

- ID
- Guid
- Title
- CategoryId
- ContentType
- StorageUrl
- SourceType
- IsAuthoritative
- IsPublished
- CreatedByUserId
- UpdatedUtc
- RowStatus
- RowVersion

AssistantKnowledgeItemVersions

Supports replace/update/versioning.

Fields:

- ID
- Guid
- KnowledgeItemId
- VersionNumber
- StorageUrl
- ExtractedText
- MetadataJson
- IsCurrent
- CreatedUtc
- CreatedByUserId
- RowStatus
- RowVersion

AssistantWorkflowTemplates

Prebuilt workflows and prompts.

AssistantWorkflowRuns

Per-user execution history for guided workflows.

AssistantUploads

Stores screenshot and file uploads linked to users and conversations.

AssistantFeedback

Stores helpfulness and comments.

AssistantAnalyticsEvents

Captures search terms, answer failures, workflow starts, etc.

Storage approach

Admins can upload PDFs, screenshots, SOPs, and internal help content.

Storage pattern

- Upload files to managed storage
- Persist file URL and metadata in database
- Extract searchable content for retrieval
- Keep original file URL for source display and future reprocessing

Recommended stored metadata

- Original file name
- Content type
- Blob / storage URL
- Uploaded by user
- Uploaded date
- Category
- Tags
- Version number
- Extracted text hash
- OCR / extraction status
- Publication status

Retrieval and answering model

Answer priority

1. Use trusted admin knowledge first.
2. Use workflow templates / playbooks next.
3. Use generated suggestions only when trusted knowledge is incomplete.

Answer presentation

Each answer should clearly label content as:

- **Trusted knowledge answer**
- **Generated suggestion**
- **Admin workflow guide**

Source display

Show:

- Source title
- Source type
- Last updated date
- Confidence / grounding indicator

Role-based behavior

Optional but recommended.

Admin

- More management answers
- Access to content and analytics tools
- More configuration-related guidance

Project manager

- Reporting, workflow, permissions, operational summaries

General user

- Simplified help
- Common tasks

- Clear guided steps

Suggested quick actions

On dashboard show quick actions like:

- Start a new guided workflow
- Explain this screen
- Help me troubleshoot
- Find a workflow guide
- Build a checklist for this task
- Show recent release help
- Find admin documentation

Empty states and onboarding

First-use experience

- Welcome message
- Explain beginner/expert mode
- Offer starter questions
- Offer workflow selection
- Show example screenshots the user can upload

Example starter questions

- How do I create a quote from an enquiry?
- Why can't I move this record to the next status?
- How do I find invoice automation issues?
- Where do I update permissions?
- What does this page do?

Design direction

Visual style

- Premium but practical
- Clean cards and panels
- Clear visual hierarchy
- Friendly but professional language
- Minimal clutter
- Strong empty states and guidance prompts

Audience fit

Should feel approachable for non-technical construction teams while still powerful enough for operational admins.

UX principles

- Fast to first answer
- Clear distinction between trusted content and generated content
- Small, guided steps over walls of text
- Actionable outputs over abstract explanations

Suggested implementation phases

Phase 1 — Foundations

- Public landing page
- Auth flow
- Assistant dashboard shell
- Conversation persistence
- Basic chat experience
- Beginner / expert toggle

- Saved conversations
- Bookmarks

Phase 2 — Knowledge grounding

- Knowledge base area
- Admin upload manager
- Storage-backed document ingestion
- Source citation display
- Category / FAQ / workflow guide structure

Phase 3 — Workflow guidance

- Prebuilt workflows
- Clarifying question engine
- Checklist outputs
- Save as playbook
- Featured workflow library

Phase 4 — Screenshot help

- Screenshot upload
- Vision analysis
- Contextual screen explanation
- Guidance response format for UI screenshots

Phase 5 — Admin intelligence

- Analytics dashboard
- Failed answer reporting
- Content gap detection
- Quality review workflow

Phase 6 — Advanced enhancements

- Role-based assistant behavior
- Multi-language support
- Email summaries
- Interactive “show me how” mode

Recommended MVP

If you want a sensible first deliverable, the MVP should include:

- Public landing page
- Secure login
- Chat dashboard
- Conversation history
- Beginner / expert mode
- Knowledge base search
- Admin content upload and organisation
- Source citations
- Bookmarks
- Prebuilt workflow prompts
- Screenshot upload and explanation
- Feedback buttons
- Admin analytics for top questions and failed answers

Example user journeys

Journey 1: New user learns CymBuild

1. Lands on public page.
2. Understands assistant value.
3. Logs in.
4. Asks “How do I create an enquiry?”

5. Gets beginner mode step-by-step answer.
6. Saves answer as bookmark.

Journey 2: Project manager troubleshoots a workflow

1. Logs in.
2. Chooses troubleshooting workflow.
3. Types “Why can’t this quote be sent?”
4. Assistant asks clarifying questions.
5. Returns checklist and likely causes.
6. Shows trusted source articles.

Journey 3: Admin improves content

1. Opens admin dashboard.
2. Sees high-volume unanswered topic.
3. Uploads new SOP and marks it authoritative.
4. Adds tags and category.
5. Reviews future answer quality improvements.

Journey 4: Screenshot assistance

1. User uploads screenshot.
2. Assistant identifies screen and likely section meanings.
3. Assistant explains what to click next and what to verify.
4. User saves as playbook.

CymBuild-specific fit

This assistant should be built as a first-class CymBuild capability, not a disconnected chatbot.

It should align with:

- Blazor WebAssembly PWA frontend
- gRPC API services
- SQL Server persistence
- CymBuild metadata and identity rules
- Role-aware enterprise operation
- Durable auditability

The assistant should help users across the core lifecycle:

- Account
- Enquiry
- EnquiryService
- Quote
- QuoteItem
- Job
- Activity / Milestone
- InvoiceSchedule
- InvoiceRequest
- InvoiceBatch

It should also help users understand workflow-driven behaviors, permissions, reporting, troubleshooting, and setup.

Final product outcome

The finished product should feel like:

The easiest way for a CymBuild user to learn faster, get unstuck, and complete tasks with confidence.

It should combine trusted internal knowledge, structured guidance, screenshot understanding, and persistent assistant memory into one polished experience.

Recommended next build step

Turn this blueprint into:

1. A full **Blazor page map and component structure**
2. A **SQL schema and EntityType/DataObjects plan**
3. A **gRPC contract set**
4. A **first implementation of landing page + authenticated dashboard**

That would provide the production-ready foundation for the assistant inside CymBuild.

CymBuild AI Assistant — Implementation Pack

Purpose

This document converts the product blueprint into a build-ready implementation pack for CymBuild. It is designed to align with:

- Blazor WebAssembly PWA
- Concurrency_API gRPC service layer
- SQL Server platform database
- CymBuild metadata and DataObjects rules
- Role-aware enterprise behaviour
- Auditability and future extensibility

This pack deliberately avoids guessing unknown repository internals and instead defines a safe, production-aligned structure that can be implemented inside the existing CymBuild platform.

1. Implementation objectives

Primary objective

Build a CymBuild AI assistant that works as both:

- an authenticated in-product copilot
- a standalone knowledge/help centre

Delivery goals

- Users can ask CymBuild questions in natural language
- Users can receive grounded, source-backed answers
- Users can follow guided workflows and save playbooks
- Users can upload screenshots for contextual help
- Admins can manage content, workflows, and quality analytics
- All important data is persisted with user ownership and auditability

2. Scope breakdown

MVP scope

The first production release should include:

- Public landing page
- Authenticated assistant dashboard
- Chat experience with streaming responses
- Saved conversations
- Bookmarks
- Beginner mode / Expert mode
- Searchable knowledge base
- Admin knowledge upload and management
- Source-backed answers
- Workflow templates
- Screenshot upload and explanation
- Feedback buttons
- Admin analytics for top questions and failures

Post-MVP scope

- Role-adaptive answer behaviour
- Multi-language support
- Email summaries

- Interactive “show me how” mode
- Screenshot region highlighting
- Knowledge gap recommendations
- Guided in-app step runners

3. Recommended information architecture

Public pages

3.1 /assistant

Landing page.

3.2 /assistant/login

Authentication entry point.

3.3 /assistant/signup

Optional signup or identity handoff entry point depending on CymBuild auth strategy.

Authenticated pages

3.4 /assistant/app

Main assistant dashboard.

3.5 /assistant/app/chat/{conversationGuid}

Conversation detail / continue thread.

3.6 /assistant/app/bookmarks

Saved answers and user bookmarks.

3.7 /assistant/app/playbooks

User playbooks and reusable workflow guides.

3.8 /assistant/app/knowledge

Searchable knowledge base.

3.9 /assistant/app/knowledge/{knowledgeItemGuid}

Knowledge item/article detail.

3.10 /assistant/app/uploads

User upload history and screenshot/doc uploads.

3.11 /assistant/app/settings

User preferences.

Admin pages

3.12 /assistant/admin

Admin dashboard home.

3.13 /assistant/admin/content

Knowledge content manager.

3.14 /assistant/admin/content/{knowledgeItemGuid}

Knowledge item detail/version manager.

3.15 /assistant/admin/workflows

Workflow template manager.

3.16 /assistant/admin/analytics

Analytics, top questions, failures, and content gaps.

3.17 /assistant/admin/review

Conversation quality and feedback review.

3.18 /assistant/admin/users

Assistant user and role management.

4. Blazor page/component structure

4.1 Recommended project placement

Use the main CymBuild PWA and shared component model.

Pages

Recommended under a feature folder such as:

- Pages/Assistant/
- Pages/Assistant/Admin/

Shared assistant components

Recommended under:

- Components/Assistant/
- Components/Assistant/Admin/
- Shared reusable components in Concursus.Components.Shared

Styling

Use CymBuild styling defaults and standard CSS source, consistent with your rule that styling should align with the shared CymBuild UI stylesheet.

Do not introduce Telerik dependency for new work unless explicitly required.

4.2 Page/component map

Landing experience

AssistantLandingPage.razor

Responsibilities:

- public hero/marketing page
- explain value, audience, feature highlights
- unauthenticated CTA handling

Child components:

- AssistantLandingHero
- AssistantLandingBenefits
- AssistantLandingFeatureGrid
- AssistantLandingHowItWorks
- AssistantLandingFaq
- AssistantLandingFooter

Authenticated workspace shell

AssistantAppPage.razor

Responsibilities:

- shell layout after auth
- left nav, centre workspace, right contextual panel
- route-aware content container

Child components:

- AssistantSideNav
- AssistantWorkspaceHeader
- AssistantQuickActionsPanel
- AssistantRecentTopicsPanel
- AssistantContextPanel

Chat workspace

AssistantChatPage.razor

Responsibilities:

- chat thread display
- prompt input
- streaming assistant replies
- beginner/expert mode toggle
- source rendering
- follow-up suggestions
- feedback actions

Child components:

- AssistantConversationList
- AssistantChatWindow
- AssistantMessageBubble
- AssistantPromptBox
- AssistantModeToggle
- AssistantSourceList
- AssistantFollowUpSuggestions
- AssistantAnswerFeedbackBar

Bookmarks

AssistantBookmarksPage.razor

Responsibilities:

- list bookmarks
- filter by tag/date
- open original answer
- delete/archive bookmark

Child components:

- AssistantBookmarkList
- AssistantBookmarkCard

Playbooks

AssistantPlaybooksPage.razor

Responsibilities:

- show saved user playbooks
- show featured admin playbooks
- open/edit/duplicate/save

Child components:

- AssistantPlaybookList
- AssistantPlaybookCard
- AssistantPlaybookStepList

Knowledge base

AssistantKnowledgePage.razor

Responsibilities:

- search and browse knowledge items
- filter by category/tag/type
- surface featured guides and FAQs

Child components:

- AssistantKnowledgeSearch
- AssistantKnowledgeCategoryList
- AssistantKnowledgeCard
- AssistantKnowledgeFilters

AssistantKnowledgeDetailPage.razor

Responsibilities:

- display article/guide detail

- show related content
- show source metadata/version info

Screenshot/upload help

AssistantUploadsPage.razor

Responsibilities:

- show user uploads
- attach screenshot into new/existing conversation
- display processing status

Child components:

- AssistantUploadDropzone
- AssistantUploadHistory
- AssistantUploadPreview

Settings

AssistantSettingsPage.razor

Responsibilities:

- mode defaults
- email summary options
- language preference
- saved preference profile

Admin dashboard

AssistantAdminDashboardPage.razor

Responsibilities:

- admin overview
- top questions
- content gaps
- recent feedback trend
- quick actions for content/workflow management

Child components:

- AssistantAdminMetricCard
- AssistantAdminRecentFailures
- AssistantAdminTopQuestions
- AssistantAdminContentGapList

Admin content manager

AssistantAdminContentPage.razor

Responsibilities:

- upload knowledge content
- replace/update versions
- category/tag management
- publication status
- authoritative source marker

Child components:

- AssistantKnowledgeUploadForm
- AssistantKnowledgeItemGrid
- AssistantKnowledgeVersionList
- AssistantKnowledgeMetadataEditor

Admin workflow manager

AssistantAdminWorkflowsPage.razor

Responsibilities:

- create/edit workflow templates
- publish/feature workflows
- classify by audience and purpose

Child components:

- AssistantWorkflowTemplateGrid
- AssistantWorkflowTemplateEditor

Admin analytics

AssistantAdminAnalyticsPage.razor

Responsibilities:

- most common questions
- failed answers
- feedback metrics
- content usage analytics
- knowledge gap clusters

Child components:

- AssistantAnalyticsTopQuestions
- AssistantAnalyticsFailureGrid
- AssistantAnalyticsContentGapGrid
- AssistantAnalyticsSourceUsageGrid

Admin review

AssistantAdminReviewPage.razor

Responsibilities:

- review poor conversations
- inspect user feedback
- mark issues and improvement actions

Child components:

- AssistantConversationReviewList
- AssistantConversationReviewDetail

5. UX behaviour contract

5.1 Chat answers

Each assistant answer should support:

- markdown or rich text body
- source list
- confidence label / answer type label
- quick actions
- bookmark action
- feedback action
- “turn into checklist” action
- “save as playbook” action

5.2 Modes

Beginner mode

- step-by-step by default
- lower jargon
- more context
- more warnings / common mistakes

Expert mode

- concise and faster
- direct action-first answers
- less explanatory overhead

5.3 Answer labels

Each answer should clearly distinguish:

- Trusted knowledge answer
- Admin workflow guide
- Generated suggestion

5.4 Suggested follow-ups

Every answer should optionally include up to 3 follow-up prompts.

6. Recommended domain model

Important CymBuild rule

Every major record must have a corresponding row in `SCore.DataObjects` created at insert time with the correct `EntityType` populated.

For this assistant, use a dedicated schema.

Recommended schema

- SAI

7. Recommended SQL entities

Below is the recommended major-entity set. These should each be treated as major records and therefore should have `SCore.DataObjects` rows created at insert time.

7.1 SAI.AssistantConversations

Represents a single conversation thread.

Suggested columns:

- ID INT IDENTITY PRIMARY KEY
- RowStatus TINYINT NOT NULL
- RowVersion ROWVERSION NOT NULL
- Guid UNIQUEIDENTIFIER NOT NULL
- UserId INT NOT NULL
- Title NVARCHAR(250) NOT NULL
- ModeCode NVARCHAR(20) NOT NULL
- LanguageCode NVARCHAR(20) NULL
- LastActivityUtc DATETIME2(7) NOT NULL
- IsPinned BIT NOT NULL
- IsArchived BIT NOT NULL
- StartedFromWorkflowTemplateId INT NULL
- LastMessageId INT NULL

Notes:

- Guid must FK to `SCore.DataObjects(Guid)`
- UserId links to `SCore.Identities(ID)`

7.2 SAI.AssistantMessages

Represents a message in a conversation.

Suggested columns:

- ID
- RowStatus
- RowVersion
- Guid
- ConversationId
- UserId

- MessageRoleCode NVARCHAR(20) NOT NULL -- user / assistant / system
- AnswerTypeCode NVARCHAR(30) NULL -- trusted / generated / workflow
- ContentMarkdown NVARCHAR(MAX) NOT NULL
- ContentPlainText NVARCHAR(MAX) NULL
- SourcePayloadJson NVARCHAR(MAX) NULL
- FollowUpPayloadJson NVARCHAR(MAX) NULL
- ConfidenceScore DECIMAL(5,4) NULL
- CreatedUtc DATETIME2(7) NOT NULL
- PromptTokens INT NULL
- CompletionTokens INT NULL
- ModelCode NVARCHAR(100) NULL
- ParentMessageId INT NULL

7.3 SAI.AssistantBookmarks

Stores saved answers.

Suggested columns:

- ID
- RowStatus
- RowVersion
- Guid
- UserId
- ConversationId
- MessageId
- Title NVARCHAR(250) NOT NULL
- Notes NVARCHAR(MAX) NULL
- TagsJson NVARCHAR(MAX) NULL
- CreatedUtc DATETIME2(7) NOT NULL

7.4 SAI.AssistantPlaybooks

Stores reusable saved workflow/help guides.

Suggested columns:

- ID
- RowStatus
- RowVersion
- Guid
- UserId NULL -- null can be used only if you explicitly want global admin-owned records via CreatedByUserId instead
- Title NVARCHAR(250) NOT NULL
- Summary NVARCHAR(1000) NULL
- PlaybookTypeCode NVARCHAR(30) NOT NULL
- VisibilityCode NVARCHAR(20) NOT NULL -- private / admin_featured / shared
- SourceConversationId INT NULL
- SourceWorkflowRunId INT NULL
- CreatedByUserId INT NOT NULL
- CreatedUtc DATETIME2(7) NOT NULL
- UpdatedUtc DATETIME2(7) NULL
- IsFeatured BIT NOT NULL

7.5 SAI.AssistantPlaybookSteps

Stores ordered steps within a playbook.

Suggested columns:

- ID
- RowStatus
- RowVersion
- Guid

- PlaybookId
- StepOrder INT NOT NULL
- Title NVARCHAR(250) NOT NULL
- InstructionMarkdown NVARCHAR(MAX) NOT NULL
- IsOptional BIT NOT NULL
- ExpectedOutcome NVARCHAR(1000) NULL

7.6 SAI.AssistantKnowledgeItems

Represents a logical knowledge asset.

Suggested columns:

- ID
- RowStatus
- RowVersion
- Guid
- Title NVARCHAR(500) NOT NULL
- Slug NVARCHAR(500) NOT NULL
- KnowledgeCategoryId INT NULL
- ContentTypeCode NVARCHAR(30) NOT NULL -- faq / guide / sop / pdf / screenshot / article
- SourceTypeCode NVARCHAR(30) NOT NULL -- upload / manual / imported
- StorageUrl NVARCHAR(1000) NOT NULL
- PreviewUrl NVARCHAR(1000) NULL
- Summary NVARCHAR(MAX) NULL
- IsAuthoritative BIT NOT NULL
- IsPublished BIT NOT NULL
- PublishedUtc DATETIME2(7) NULL
- CreatedByUserId INT NOT NULL
- UpdatedByUserId INT NULL
- CreatedUtc DATETIME2(7) NOT NULL
- UpdatedUtc DATETIME2(7) NULL

7.7 SAI.AssistantKnowledgeItemVersions

Versioned knowledge content.

Suggested columns:

- ID
- RowStatus
- RowVersion
- Guid
- KnowledgeItemId
- VersionNumber INT NOT NULL
- StorageUrl NVARCHAR(1000) NOT NULL
- ExtractedText NVARCHAR(MAX) NULL
- ExtractionStatusCode NVARCHAR(30) NOT NULL
- MetadataJson NVARCHAR(MAX) NULL
- FileHash NVARCHAR(200) NULL
- IsCurrent BIT NOT NULL
- CreatedByUserId INT NOT NULL
- CreatedUtc DATETIME2(7) NOT NULL

7.8 SAI.AssistantKnowledgeCategories

Knowledge base categories.

Suggested columns:

- ID
- RowStatus
- RowVersion
- Guid

- Name NVARCHAR(250) NOT NULL
- Code NVARCHAR(50) NOT NULL
- Description NVARCHAR(1000) NULL
- DisplayOrder INT NOT NULL
- IsVisible BIT NOT NULL

7.9 SAi.AssistantKnowledgeTags

Reusable tags.

Suggested columns:

- ID
- RowStatus
- RowVersion
- Guid
- Name NVARCHAR(100) NOT NULL
- Code NVARCHAR(50) NOT NULL

7.10 SAi.AssistantKnowledgeItemTags

Junction table.

Suggested columns:

- ID
- RowStatus
- RowVersion
- Guid
- KnowledgeItemId
- KnowledgeTagId

7.11 SAi.AssistantUploads

Tracks uploaded files and screenshots.

Suggested columns:

- ID
- RowStatus
- RowVersion
- Guid
- UserId
- ConversationId NULL
- KnowledgeItemId NULL
- StorageUrl NVARCHAR(1000) NOT NULL
- FileName NVARCHAR(500) NOT NULL
- ContentType NVARCHAR(200) NOT NULL
- FileSizeBytes BIGINT NOT NULL
- UploadPurposeCode NVARCHAR(30) NOT NULL -- screenshot / knowledge / attachment
- ProcessingStatusCode NVARCHAR(30) NOT NULL
- VisionSummary NVARCHAR(MAX) NULL
- CreatedUtc DATETIME2(7) NOT NULL

7.12 SAi.AssistantWorkflowTemplates

Prebuilt workflows.

Suggested columns:

- ID
- RowStatus
- RowVersion
- Guid
- Code NVARCHAR(50) NOT NULL
- Title NVARCHAR(250) NOT NULL

- Summary NVARCHAR(1000) NULL
- AudienceCode NVARCHAR(30) NULL
- TemplatePrompt NVARCHAR(MAX) NOT NULL
- ClarificationSchemaJson NVARCHAR(MAX) NULL
- OutputFormatCode NVARCHAR(30) NOT NULL
- IsPublished BIT NOT NULL
- IsFeatured BIT NOT NULL
- CreatedByUserId INT NOT NULL
- CreatedUtc DATETIME2(7) NOT NULL
- UpdatedUtc DATETIME2(7) NULL

7.13 SAi.AssistantWorkflowRuns

Tracks a user execution of a workflow.

Suggested columns:

- ID
- RowStatus
- RowVersion
- Guid
- UserId
- WorkflowTemplateId
- ConversationId NULL
- StatusCode NVARCHAR(30) NOT NULL
- InputJson NVARCHAR(MAX) NULL
- OutputJson NVARCHAR(MAX) NULL
- StartedUtc DATETIME2(7) NOT NULL
- CompletedUtc DATETIME2(7) NULL

7.14 SAi.AssistantWorkflowRunSteps

Step progress for guided workflows.

Suggested columns:

- ID
- RowStatus
- RowVersion
- Guid
- WorkflowRunId
- StepOrder
- Title
- InstructionMarkdown
- StatusCode NVARCHAR(30) NOT NULL
- CompletedUtc DATETIME2(7) NULL

7.15 SAi.AssistantFeedback

Feedback on assistant answers.

Suggested columns:

- ID
- RowStatus
- RowVersion
- Guid
- UserId
- ConversationId
- MessageId
- FeedbackCode NVARCHAR(20) NOT NULL -- helpful / unhelpful
- Comment NVARCHAR(MAX) NULL
- CreatedUtc DATETIME2(7) NOT NULL

7.16 SAi.AssistantAnalyticsEvents

Event-level analytics.

Suggested columns:

- ID
- RowStatus
- RowVersion
- Guid
- UserId NULL
- ConversationId NULL
- EventTypeCode NVARCHAR(50) NOT NULL
- EventUtc DATETIME2(7) NOT NULL
- TopicText NVARCHAR(1000) NULL
- PayloadJson NVARCHAR(MAX) NULL
- SuccessFlag BIT NULL

7.17 SAi.AssistantContentGaps

Admin-reviewable gap tracking.

Suggested columns:

- ID
- RowStatus
- RowVersion
- Guid
- Title NVARCHAR(500) NOT NULL
- Description NVARCHAR(MAX) NULL
- TopicCluster NVARCHAR(250) NULL
- OccurrenceCount INT NOT NULL
- LastSeenUtc DATETIME2(7) NOT NULL
- StatusCode NVARCHAR(30) NOT NULL -- new / reviewing / addressed
- SuggestedKnowledgeItemId INT NULL

8. EntityType and DataObjects plan

8.1 Required rule

For each major table above, create a matching SCore.EntityTypes entry and ensure inserts create SCore.DataObjects rows at insert time with EntityTypeId populated.

8.2 Recommended major entity types

At minimum, create EntityTypes for:

- Assistant Conversations
- Assistant Messages
- Assistant Bookmarks
- Assistant Playbooks
- Assistant Playbook Steps
- Assistant Knowledge Items
- Assistant Knowledge Item Versions
- Assistant Knowledge Categories
- Assistant Knowledge Tags
- Assistant Knowledge Item Tags
- Assistant Uploads
- Assistant Workflow Templates
- Assistant Workflow Runs
- Assistant Workflow Run Steps
- Assistant Feedback
- Assistant Analytics Events
- Assistant Content Gaps

8.3 Upsert pattern

Follow CymBuild convention:

- create DataObjects row first at insert time using the final Guid
- set EntityTypeId immediately
- avoid insert-then-update patterns

If you choose a stored procedure-based persistence strategy, implement explicit upsert procedures per major table or a consistent service-level insert orchestration.

9. Workflow and status considerations

9.1 Direct status warning

Do not directly update workflow statuses when a workflow-driven entity exists.

9.2 Where workflow is needed

Most assistant tables can begin without full workflow.

Recommended workflow-enabled entities for later phases:

- AssistantKnowledgeItems
- AssistantWorkflowTemplates
- AssistantContentGaps

Example workflow usage

Knowledge item lifecycle

- Draft
- Under Review
- Published
- Retired

Content gap lifecycle

- New
- Investigating
- Assigned
- Addressed
- Closed

If workflow is implemented, use `SCore.DataObjectTransitionUpsert` and latest transition as source of truth.

10. Storage and upload design

10.1 Upload handling requirements

Admins can upload:

- PDFs
- screenshots
- SOPs
- internal product docs
- workflow guides

Users can upload:

- screenshots
- supporting files for help requests

10.2 Storage pattern

Recommended approach:

1. upload binary file to managed storage
2. persist file metadata and storage URL in `SAi.AssistantUploads`
3. create or update `SAi.AssistantKnowledgeItems` when used as knowledge content
4. extract searchable text into `AssistantKnowledgeItemVersions.ExtractedText`
5. use stored URL for rendering and citation

10.3 Processing lifecycle

Suggested upload statuses:

- PendingUpload
- Uploaded
- QueuedForExtraction
- Extracted
- ExtractionFailed
- Published

11. Retrieval and answer orchestration

11.1 Answer priority order

1. authoritative admin content
2. published workflow templates/playbooks
3. other published knowledge content
4. generated answer synthesis

11.2 Required answer payload

Every answer payload should support:

- final answer body
- answer type label
- sources list
- confidence score
- follow-up suggestions
- optional checklist version
- optional workflow version

11.3 Suggested source object

Each source in the response should include:

- title
- type
- url
- knowledge item guid
- version number
- excerpt
- is authoritative flag

12. gRPC contract design

12.1 Services overview

Recommended service set:

- AssistantChatService
- AssistantConversationService
- AssistantKnowledgeService
- AssistantWorkflowService
- AssistantBookmarkService
- AssistantUploadService
- AssistantAdminService
- AssistantAnalyticsService

12.2 AssistantChatService

Responsibilities

- send user prompt
- stream assistant response
- attach sources
- apply mode and workflow context
- support screenshot-linked prompts

Suggested operations

SendMessage

Request:

- conversationGuid
- userMessage
- modeCode
- workflowTemplateGuid optional
- attachedUploadGuids
- languageCode optional

Response:

- assistantMessageGuid
- answerTypeCode
- content chunks or final content
- sources
- followUpSuggestions
- confidenceScore

RegenerateAnswer

Request:

- conversationGuid
- parentMessageGuid
- modeCode

ConvertAnswerToChecklist

Request:

- messageGuid

SaveAnswerAsPlaybook

Request:

- messageGuid
- title
- visibilityCode

12.3 AssistantConversationService

Suggested operations

- CreateConversation
- GetConversation
- ListConversationsForUser
- RenameConversation
- ArchiveConversation
- DeleteConversation
- GetConversationMessages

12.4 AssistantKnowledgeService

Suggested operations

- SearchKnowledge
- GetKnowledgeItem
- GetKnowledgeCategories
- GetFeaturedKnowledge
- GetRelatedKnowledge

For admin:

- CreateKnowledgeItem
- UpdateKnowledgeItem
- PublishKnowledgeItem
- ReplaceKnowledgeItemVersion
- ListKnowledgeVersions

12.5 AssistantWorkflowService

Suggested operations

- ListWorkflowTemplates
- GetWorkflowTemplate
- StartWorkflowRun
- AdvanceWorkflowRun
- CompleteWorkflowRun
- ListPlaybooks
- GetPlaybook

Admin:

- CreateWorkflowTemplate
- UpdateWorkflowTemplate
- PublishWorkflowTemplate
- FeatureWorkflowTemplate

12.6 AssistantBookmarkService

Suggested operations

- CreateBookmark
- ListBookmarks
- DeleteBookmark
- UpdateBookmark

12.7 AssistantUploadService

Suggested operations

- CreateUploadSession
- CompleteUpload
- ListUserUploads
- AnalyzeScreenshot
- AttachUploadToConversation

Screenshot analysis response should include

- detected screen summary
- notable sections
- likely next actions
- possible mistakes / missing setup
- confidence indicators

12.8 AssistantAdminService

Suggested operations

- GetAdminDashboardSummary
- ListFeedback
- ListFailedAnswers
- ListContentGaps
- AssignContentGap
- ResolveContentGap

12.9 AssistantAnalyticsService

Suggested operations

- GetTopQuestions
- GetTopicTrends
- GetFailedAnswerSummary
- GetSourceUsageSummary

- GetWorkflowUsageSummary

13. DTO and response design guidance

13.1 Message DTO

Suggested payload fields:

- Guid
- RoleCode
- AnswerTypeCode
- ContentMarkdown
- CreatedUtc
- ConfidenceScore
- Sources[]
- FollowUps[]
- FeedbackState

13.2 Knowledge item DTO

Suggested payload fields:

- Guid
- Title
- Summary
- Category
- Tags[]
- ContentTypeCode
- StorageUrl
- PreviewUrl
- IsAuthoritative
- LastUpdatedUtc

14. Repository and service layering

14.1 Recommended backend layering

Use the normal CymBuild separation:

- gRPC contract layer
- application/service orchestration
- repository/data access layer
- SQL stored procedures / queries

14.2 Suggested service classes

- AssistantChatOrchestrator
- AssistantConversationManager
- AssistantKnowledgeManager
- AssistantWorkflowManager
- AssistantUploadManager
- AssistantAnalyticsManager
- AssistantAdminReviewManager

14.3 Suggested repository classes

- AssistantConversationRepository
- AssistantMessageRepository
- AssistantKnowledgeRepository
- AssistantUploadRepository
- AssistantWorkflowRepository
- AssistantBookmarkRepository
- AssistantFeedbackRepository
- AssistantAnalyticsRepository

15. Metadata-driven opportunities

The assistant itself can begin with a custom polished UI, but admin management areas should use metadata-driven behaviour where practical.

Recommended metadata-driven candidates

- Admin content grids
- Workflow template grids
- Feedback review grids
- Analytics result grids where supported
- Knowledge metadata forms

Recommended custom UI candidates

- Public landing page
- Chat experience
- Conversation rendering
- Source cards
- Follow-up action bar
- Screenshot explanation experience

This respects the CymBuild rule to use metadata where it exists, while not forcing a generic edit experience onto the premium assistant front end.

16. Security and permissions

16.1 User scoping

All user-owned records must include UserId as requested.

Examples:

- conversations
- messages
- bookmarks
- workflow runs
- uploads
- feedback

16.2 Admin scoping

Admin features should require explicit role/group checks.

Suggested assistant roles:

- AssistantUser
- AssistantPowerUser
- AssistantAdmin
- AssistantContentManager
- AssistantReviewer

These can be resolved from CymBuild security groups rather than a separate auth system.

16.3 Data rules

- users must only see their own private conversations/bookmarks/uploads/playbooks unless explicitly shared
- admins can see admin review data and published knowledge content
- audit every important content/admin action

17. Analytics model

17.1 Capture events for

- search queries
- asked questions
- workflow starts/completions
- screenshot analyses
- source clicks

- bookmarks created
- feedback submitted
- answer failures / low confidence

17.2 Admin metrics to surface

- most common questions
- most common failed topics
- unanswered topic clusters
- most useful knowledge items
- low-rated answers
- top workflow templates
- screenshot-help frequency

18. Recommended implementation sequence

Sprint 1 — Platform foundations

Deliver:

- schema and entity plan
- initial SQL objects for core assistant tables
- gRPC contracts for conversations, chat, bookmarks
- public landing page
- authenticated shell page

Outcome

You can enter the assistant, create a conversation, and persist basic chat threads.

Sprint 2 — Chat and persistence

Deliver:

- conversation list and detail page
- streaming assistant UX
- message persistence
- beginner/expert mode
- bookmark creation
- follow-up suggestions UI

Outcome

Users can use the assistant as a real copilot with saved history.

Sprint 3 — Knowledge base grounding

Deliver:

- admin content upload flow
- knowledge item tables and versioning
- knowledge search page
- source rendering in answers
- article detail page

Outcome

Answers become grounded in trusted CymBuild content.

Sprint 4 — Workflow templates and playbooks

Deliver:

- workflow template CRUD
- workflow selection in assistant
- checklist outputs
- save as playbook
- playbook library page

Outcome

Assistant becomes structured and repeatable for common tasks.

Sprint 5 — Screenshot help

Deliver:

- upload flow for screenshots
- screenshot analysis pipeline
- screenshot summary answer format
- upload history page

Outcome

Users can get contextual UI help from actual CymBuild screens.

Sprint 6 — Admin review and analytics

Deliver:

- admin dashboard
- common question analytics
- failed answer review
- content gap tracking
- review workflow

Outcome

Admins can continuously improve the assistant.

19. Recommended MVP acceptance criteria

Landing page

- unauthenticated users can open it
- value proposition is clear
- login/signup buttons route correctly

Chat dashboard

- authenticated users can start a conversation
- messages persist by user
- responses stream to the UI
- user can revisit and continue a prior conversation

Knowledge grounding

- admin can upload content
- uploaded content is stored with URL + metadata
- assistant can cite knowledge sources in answers

Bookmarks and playbooks

- user can bookmark an answer
- user can save a playbook
- saved items are linked to the correct user

Screenshot help

- user can upload screenshot
- assistant returns a contextual explanation
- upload is stored and tied to the user

Admin tools

- admin can see top questions and failed answers
- admin can manage knowledge items and workflows

20. Suggested SQL build order

To stay safe and deterministic, the recommended build order is:

1. Seed EntityTypes for SAI major entities
2. Create SAI schema
3. Create base tables with explicit columns
4. Add PKs, FKs, unique indexes
5. Add DataObjects FK relationships
6. Create stored procedures / upsert patterns
7. Add read views/TVFs if needed
8. Seed workflow categories/statuses only where required
9. Seed metadata for admin grids/forms

21. Suggested SQL object naming convention

Tables

- SAI.AssistantConversations
- SAI.AssistantMessages
- SAI.AssistantKnowledgeItems

Upsert procedures

- SAI.AssistantConversationUpsert
- SAI.AssistantBookmarkUpsert
- SAI.AssistantKnowledgeItemUpsert

Reads

- SAI.tvf_AssistantConversationList
- SAI.tvf_AssistantKnowledgeSearch
- SAI.tvf_AssistantAdminDashboard

Utility / processing

- SAI.AssistantKnowledgeVersionCreate
- SAI.AssistantUploadComplete
- SAI.AssistantContentGapRecord

22. Recommended first code deliverables

The fastest meaningful production start would be to build these first:

Deliverable A — SQL foundation

- SAI schema
- core conversation/message/bookmark tables
- knowledge item tables
- workflow template tables
- upload table
- feedback/analytics tables
- EntityType/DataObjects seeding

Deliverable B — gRPC foundation

- AssistantConversationService
- AssistantChatService
- AssistantKnowledgeService
- AssistantBookmarkService

Deliverable C — UI foundation

- Landing page
- Authenticated assistant shell
- Chat page
- Bookmarks page
- Knowledge search page
- Admin content page

23. What should be built custom vs metadata-driven

Build custom first

- landing page
- assistant shell
- chat surface
- prompt box
- answer cards
- source rendering
- screenshot explanation experience

Use metadata-driven admin surfaces where appropriate

- knowledge item management grids
- workflow template management grids
- feedback review grids
- analytics result grids

This keeps the user-facing product premium and purpose-built while staying aligned with CymBuild admin conventions.

24. Risks and controls

Risk: assistant answers become too generic

Control:

- prioritise authoritative knowledge items
- show answer type labels
- surface sources on every grounded answer

Risk: content becomes stale

Control:

- versioning
- publication status
- last updated display
- admin gap analytics

Risk: users cannot trust generated output

Control:

- clearly distinguish trusted knowledge vs generated suggestion
- allow feedback on every answer

Risk: assistant data breaks CymBuild conventions

Control:

- enforce DataObjects creation at insert time
- use EntityTypes for major records
- keep auditable admin actions

25. Final implementation recommendation

Build the assistant as a dedicated SAi domain inside CymBuild with:

- a premium custom front end for chat/help experiences
- metadata-driven admin management where practical
- gRPC orchestration services
- SQL persistence following strict SCore.DataObjects rules
- source-grounded answers and strong admin analytics

This gives you a product that feels modern and user-friendly while still fitting the CymBuild platform architecture.

26. Recommended immediate next step

The next concrete build artifact should be one of these, in order:

1. **Production-ready SQL foundation** for the SAi schema and tables
2. **gRPC proto/contracts** for the core assistant services
3. **Blazor implementation** for landing page + authenticated assistant dashboard

The best place to start is the SQL foundation because it locks the persistence model, DataObjects compliance, and user ownership model before UI and service work are layered on top.

CymBuild AI Assistant — C# Implementation Plan

Purpose

This document turns the assistant SQL and gRPC contract layers into a concrete C# implementation plan for:

- Concursus_API
- Concursus_API_Client
- Concursus_EF
- Concursus.PWA

It is intentionally structured so the assistant can be implemented in normal CymBuild patterns without guessing hidden repository internals.

1. Delivery objective

Implement the assistant as a first-class CymBuild feature with:

- server-side orchestration in Concursus_API
- typed gRPC contracts and client wrappers
- SQL-backed persistence through the new SAI schema
- Blazor pages/components in Concursus.PWA
- admin list experiences that can use the metadata grid layer already seeded

2. Recommended solution structure

2.1 Concursus_API_Client

Add assistant client contracts and generated service client support.

Suggested folders

- Assistant/
- Assistant/Models/
- Assistant/Services/

Expected contents

- generated gRPC client classes from the assistant proto
- UI-facing DTO wrappers where needed
- response helpers for streaming chat messages

2.2 Concursus_API

Add assistant application services and gRPC endpoints.

Suggested folders

- Services/Assistant/
- Services/Assistant/Contracts/
- Services/Assistant/Models/
- Services/Assistant/Repositories/
- Services/Assistant/Vision/
- Services/Assistant/Analytics/

Expected contents

- gRPC endpoint classes
- orchestration services
- repository interfaces and implementations
- mapping helpers

- content retrieval helpers
- screenshot analysis adapter abstraction

2.3 Concursus_EF

Add data access support for SAI.

Suggested folders

- Repositories/Assistant/
- Models/Assistant/ only if EF entities are already used this way

Expected contents

- repository implementations that call the SQL procedures / TVFs / views
- typed read models for list/read operations
- parameterised SQL execution wrappers

2.4 Concursus.PWA

Add assistant UX pages and components.

Suggested folders

- Pages/Assistant/
- Pages/Assistant/Admin/
- Components/Assistant/
- Components/Assistant/Admin/
- Services/Assistant/

Expected contents

- landing page
- authenticated shell
- chat page
- bookmarks page
- knowledge page
- admin pages
- upload/screenshot components
- assistant client wrapper service for the Blazor app

3. Recommended server-side layering

3.1 gRPC endpoint layer

This should stay thin.

Responsibilities

- validate request shape
- call orchestration service
- map domain result to proto reply
- stream responses where needed
- translate known exceptions to RPC-friendly failures

Suggested endpoint classes

- AssistantConversationGrpcService
- AssistantChatGrpcService
- AssistantKnowledgeGrpcService
- AssistantWorkflowGrpcService
- AssistantBookmarkGrpcService
- AssistantUploadGrpcService
- AssistantAdminGrpcService
- AssistantAnalyticsGrpcService

3.2 orchestration layer

This is the main assistant application layer.

Responsibilities

- conversation lifecycle
- message persistence
- answer generation orchestration
- knowledge retrieval and ranking
- workflow run coordination
- upload routing
- screenshot analysis orchestration
- analytics and feedback handling

Suggested classes

- AssistantConversationManager
- AssistantChatOrchestrator
- AssistantKnowledgeManager
- AssistantWorkflowManager
- AssistantBookmarkManager
- AssistantUploadManager
- AssistantAdminManager
- AssistantAnalyticsManager

3.3 repository layer

Use repository classes that map cleanly to the SAI SQL objects already defined.

Suggested interfaces

- IAssistantConversationRepository
- IAssistantMessageRepository
- IAssistantBookmarkRepository
- IAssistantKnowledgeRepository
- IAssistantWorkflowRepository
- IAssistantUploadRepository
- IAssistantFeedbackRepository
- IAssistantAnalyticsRepository
- IAssistantContentGapRepository

Suggested implementations

- AssistantConversationRepository
- AssistantMessageRepository
- AssistantBookmarkRepository
- AssistantKnowledgeRepository
- AssistantWorkflowRepository
- AssistantUploadRepository
- AssistantFeedbackRepository
- AssistantAnalyticsRepository
- AssistantContentGapRepository

4. Mapping the SQL layer to repositories

The current SQL build includes:

- create/upsert stored procedures
- read TVFs/views
- admin metadata seed

That means the repository layer should primarily use:

- stored procedures for writes
- TVFs/views for reads

4.1 Conversation repository

Write methods

- CreateConversationAsync
- RenameConversationAsync
- ArchiveConversationAsync

Read methods

- GetConversationAsync
- ListConversationsForUserAsync
- GetConversationMessagesAsync

SQL alignment

- SAI.AssistantConversationUpsert
- SAI.tvf_AssistantConversationList
- SAI.tvf_AssistantConversationMessages

4.2 Message repository

Write methods

- CreateMessageAsync

Read methods

- message reads can flow through conversation read methods for now

SQL alignment

- SAI.AssistantMessageCreate

4.3 Bookmark repository

Write methods

- CreateBookmarkAsync
- UpdateBookmarkAsync
- DeleteBookmarkAsync

Read methods

- ListBookmarksAsync

SQL alignment

- SAI.AssistantBookmarkUpsert
- add bookmark list/read SQL object next if not already present

4.4 Knowledge repository

Write methods

- CreateKnowledgeItemAsync
- UpdateKnowledgeItemAsync
- CreateKnowledgeVersionAsync
- PublishKnowledgeItemAsync

Read methods

- SearchKnowledgeAsync
- GetKnowledgeItemAsync
- ListKnowledgeVersionsAsync
- GetKnowledgeCategoriesAsync

SQL alignment

- SAI.AssistantKnowledgeItemUpsert
- SAI.AssistantKnowledgeItemVersionCreate
- SAI.tvf_AssistantKnowledgeSearch

4.5 Workflow repository

Write methods

- CreateWorkflowTemplateAsync
- UpdateWorkflowTemplateAsync
- CreateWorkflowRunAsync
- UpdateWorkflowRunAsync

Read methods

- ListWorkflowTemplatesAsync
- GetWorkflowTemplateAsync
- ListPlaybooksAsync
- GetPlaybookAsync

SQL alignment

- SAi.AssistantWorkflowTemplateUpsert
- SAi.AssistantWorkflowRunUpsert

4.6 Upload repository

Write methods

- CreateUploadAsync
- AttachUploadToConversationAsync

Read methods

- ListUserUploadsAsync
- GetUploadAsync

SQL alignment

- SAi.AssistantUploadCreate

4.7 Feedback repository

Write methods

- CreateFeedbackAsync

Read methods

- ListFeedbackAsync
- ListFailedAnswersAsync

SQL alignment

- SAi.AssistantFeedbackCreate
- add list/read SQL object next if needed

4.8 Analytics repository

Write methods

- CreateAnalyticsEventAsync
- UpsertContentGapAsync

Read methods

- GetDashboardSummaryAsync
- GetTopQuestionsAsync
- GetTopicTrendsAsync
- GetSourceUsageSummaryAsync
- GetWorkflowUsageSummaryAsync

SQL alignment

- SAi.AssistantAnalyticsEventCreate
- SAi.AssistantContentGapUpsert
- SAi.vw_AssistantAdminDashboard

5. Recommended C# model split

5.1 Domain/application models

Keep internal application models separate from proto-generated types.

Suggested internal models

- AssistantConversation
- AssistantMessage
- AssistantSource
- AssistantFollowUpSuggestion
- AssistantBookmark
- AssistantKnowledgeItem
- AssistantKnowledgeVersion
- AssistantWorkflowTemplate
- AssistantWorkflowRun
- AssistantUpload
- AssistantFeedback
- AssistantContentGap
- AssistantDashboardSummary

Reason

This avoids polluting orchestration code with gRPC transport concerns.

5.2 SQL read models

Use slim read models when pulling from TVFs/views.

Suggested read models

- AssistantConversationListRow
- AssistantConversationMessageRow
- AssistantKnowledgeSearchRow
- AssistantAdminDashboardRow
- AssistantFeedbackListRow
- AssistantContentGapListRow

5.3 Proto mapping layer

Add explicit mappers:

- AssistantProtoMapper
- AssistantEnumMapper

Responsibilities

- domain to proto
- proto to domain request models
- enum conversion
- safe null handling for wrappers/timestamps

6. Recommended chat orchestration design

6.1 AssistantChatOrchestrator flow

SendMessageAsync

1. Resolve user context and validate conversation access.
2. Persist the user message through AssistantMessageCreate.
3. Resolve conversation mode and optional workflow context.
4. Retrieve relevant knowledge items.
5. Build answer context object.
6. Call answer engine adapter.
7. Stream chunks back to caller.
8. Persist final assistant message with sources/follow-ups.
9. Record analytics event.

Important split

The answer engine should be abstracted behind an interface.

Suggested interface

`IAssistantAnswerEngine`

Methods

- `StreamAnswerAsync`
- `ConvertToChecklistAsync`
- `GenerateFollowUpsAsync`

This avoids hard-binding the assistant to a single LLM provider inside your business logic.

6.2 Source-grounding flow

`AssistantKnowledgeManager.SearchRelevantSourcesAsync`

Input:

- user question
- workflow context
- mode
- attached screenshot/upload context

Output:

- ranked `AssistantKnowledgeItem` list
- excerpts
- authoritative indicators

Ranking order

1. authoritative published knowledge
2. published workflow guides
3. other published knowledge
4. fallback generated answer if knowledge is weak

6.3 Suggested answer context model

Create an internal type:

`AssistantAnswerContext`

Fields:

- conversation
- current user message
- recent conversation messages
- selected workflow template
- uploaded screenshot summaries
- ranked knowledge sources
- mode
- language

7. Screenshot / upload implementation design

7.1 Upload lifecycle

Suggested flow

1. PWA requests upload session.
2. API returns upload target.
3. Client uploads file to storage.
4. Client calls `CompleteUpload`.
5. API stores upload metadata in `SAi.AssistantUploads`.
6. Optional background extraction/vision analysis occurs.

7.2 Screenshot analysis abstraction

Use an adapter rather than binding screenshot analysis directly in the gRPC endpoint.

Suggested interface

IAssistantVisionAnalyzer

Methods

- AnalyzeScreenshotAsync
- SummarizeUiScreenAsync

Return type

AssistantScreenshotAnalysisResult

Fields:

- detected screen summary
- notable sections
- likely next actions
- possible mistakes
- confidence score

8. Admin service implementation design

8.1 AssistantAdminManager

Responsibilities

- dashboard summary
- feedback review
- failed answer review
- content gap review
- admin moderation actions

Suggested methods

- GetDashboardSummaryAsync
- ListFeedbackAsync
- ListFailedAnswersAsync
- ListContentGapsAsync
- AssignContentGapAsync
- ResolveContentGapAsync

8.2 Content gap handling

Use a deterministic path:

- failed feedback or low-confidence answer raises analytics event
- analytics processor groups into topic clusters
- admin reviews and links a suggested knowledge item
- content gap status updates through repository

9. Analytics service implementation design

9.1 AssistantAnalyticsManager

Responsibilities

- write assistant events
- produce top questions/trends
- source usage analysis
- workflow usage metrics

Suggested methods

- TrackQuestionAskedAsync
- TrackBookmarkCreatedAsync
- TrackWorkflowStartedAsync
- TrackScreenshotAnalysedAsync
- GetTopQuestionsAsync
- GetTopicTrendsAsync

- GetSourceUsageSummaryAsync
- GetWorkflowUsageSummaryAsync

9.2 Event capture points

Track analytics during:

- conversation creation
- user question send
- answer completion
- bookmark save
- playbook save
- workflow start/complete
- screenshot analysis
- feedback submission
- source click

10. Suggested exception model

Use explicit assistant exceptions instead of generic thrown strings.

Suggested exception types

- AssistantConversationNotFoundException
- AssistantAccessDeniedException
- AssistantKnowledgeItemNotFoundException
- AssistantWorkflowTemplateNotFoundException
- AssistantUploadNotFoundException
- AssistantValidationException
- AssistantAnswerGenerationException

gRPC mapping

Map these to:

- NotFound
- PermissionDenied
- InvalidArgument
- FailedPrecondition
- Internal

11. Suggested DI registrations

11.1 API service registrations

Add registrations such as:

- IAssistantConversationRepository
- IAssistantMessageRepository
- IAssistantBookmarkRepository
- IAssistantKnowledgeRepository
- IAssistantWorkflowRepository
- IAssistantUploadRepository
- IAssistantFeedbackRepository
- IAssistantAnalyticsRepository
- IAssistantContentGapRepository

Managers:

- AssistantConversationManager
- AssistantChatOrchestrator
- AssistantKnowledgeManager
- AssistantWorkflowManager
- AssistantBookmarkManager
- AssistantUploadManager
- AssistantAdminManager
- AssistantAnalyticsManager

Adapters:

- IAssistantAnswerEngine
- IAssistantVisionAnalyzer

12. Recommended implementation order in C#

Phase A — proto integration

- add proto file
- generate client/server classes
- wire service registrations

Phase B — repositories

- implement repositories against SAI procedures and TVFs/views
- add read/write model mapping

Phase C — conversations and chat

- implement conversation service
- implement message persistence
- implement chat orchestrator shell with placeholder answer engine

Phase D — knowledge and workflows

- implement knowledge service
- implement workflow service
- wire source ranking

Phase E — uploads and screenshot help

- implement upload service
- implement vision analysis adapter

Phase F — admin and analytics

- implement admin summary/review endpoints
- implement analytics endpoints and tracking

13. Blazor client integration plan

13.1 Client service wrapper

Create a UI-facing client wrapper instead of calling generated gRPC clients directly from components.

Suggested interface

IAssistantClientService

Suggested methods

- CreateConversationAsync
- ListConversationsAsync
- GetConversationMessagesAsync
- SendMessageStreamAsync
- SearchKnowledgeAsync
- ListBookmarksAsync
- CreateBookmarkAsync
- ListWorkflowTemplatesAsync
- StartWorkflowRunAsync
- CreateUploadSessionAsync
- CompleteUploadAsync
- AnalyzeScreenshotAsync
- GetAdminDashboardSummaryAsync

Benefits

- hides transport details from pages

- simplifies testing
- centralises retry and exception handling

13.2 Blazor state containers

Suggested lightweight state services

- AssistantConversationState
- AssistantChatComposerState
- AssistantKnowledgeState
- AssistantAdminState

Purpose

- current conversation id
- current mode
- current workflow selection
- selected upload context
- optimistic streaming state

14. Page wiring plan for PWA

14.1 AssistantChatPage.razor

Required services

- IAssistantClientService
- user context service
- navigation manager

Flow

1. load conversation from route
2. load message history
3. allow message send
4. render streaming chunks progressively
5. show final sources and follow-ups
6. support bookmark action and playbook save

14.2 AssistantKnowledgePage.razor

Flow

- search knowledge by free text
- filter by category
- open detail view

14.3 AssistantAdminContentPage.razor

Flow

- show metadata grid using seeded grid code AIKNOWLEDGE
- add upload/create actions via custom toolbar or page buttons

14.4 AssistantAdminWorkflowsPage.razor

Flow

- show metadata grid using AIWORKFLOWS

14.5 AssistantAdminReviewPage.razor

Flow

- show metadata grid using AIFEEDBACK

14.6 AssistantAdminAnalyticsPage.razor

Flow

- show metadata grid using AICONTENTGAPS
- surface dashboard metric cards from gRPC admin/analytics endpoints

15. Suggested server-side placeholder strategy

Before connecting a full answer engine, implement a safe placeholder answer path.

Placeholder answer engine behaviour

- echo that this is a development placeholder
- summarise the top ranked knowledge item
- return deterministic follow-up suggestions
- persist answer exactly as a normal assistant message

Reason

This allows the full assistant workflow, persistence, and UI streaming to be tested before final LLM integration.

16. Test strategy

16.1 Repository tests

- verify procedure and TVF mappings
- verify GUID/user scoping
- verify create-only methods do not update existing rows

16.2 Service tests

- conversation access checks
- knowledge search prioritisation
- workflow run lifecycle
- upload-to-conversation flow
- feedback and analytics tracking

16.3 UI tests

- streaming message rendering
- empty states
- bookmark creation
- admin page loading
- upload completion path

17. Recommended concrete next artifact

The next code artifact to generate is:

Option 1

Production-ready proto-backed C# service skeletons for Concursus_API

This should include:

- endpoint classes
- manager interfaces/classes
- repository interfaces
- mapper skeletons
- DI registration example

Option 2

Production-ready Blazor assistant shell and chat page wired to the client service abstractions

The best next step is **Option 1**, because it locks the backend structure before the PWA consumes it.

18. Final implementation note

This plan preserves existing CymBuild architecture principles:

- additive domain only
- no destructive changes
- DataObjects compliance remains intact

- metadata-driven admin surfaces remain supported
- custom premium UX remains reserved for the user-facing assistant experience

It provides a clean path from the new SAI SQL domain and proto contracts into real server-side implementation without guessing unknown code that was not present in the uploaded repository/schema material.